



รายงานความก้าวหน้าโครงการ Aphrodize

ระบบวิเคราะห์ภาพใบหน้าและติดตามข้อมูลสุขภาพรายวัน

จัดทำโดย

พชรพล เกตุแก้ว 6610110190

จักรชัย ปลั่งใหม่ 6610110428

นางสาว พิชญ์ณัญญา แสนประเสริฐ 6610110579

เสนอ

ผศ.ดร. สหพงศ์ สมวงศ์

ดร. รัฐชัย วงศ์ธนวิจิต

ดร. วรินทร์ โรจนกรินทร์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 241-353 AI Ecosystem Module

ภาคเรียนที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตหาดใหญ่

สารบัญ

บทที่ 1 บทนำ	3
บทที่ 2 ทฤษฎีและเครื่องมือที่เกี่ยวข้อง	4
บทที่ 3 การออกแบบและพัฒนาระบบ	17
บทที่ 4 ผลการพัฒนาและการทดสอบ	22
บทที่ 5 สรุปและข้อเสนอแนะ	29
เอกสารอ้างอิง	32
ภาคผนวก หลักฐานและตำแหน่งโค้ด	34

บทที่ 1 บทนำ

1 1 ที่มาและความสำคัญ

Aphrodize

เป็นโครงการวิเคราะห์และติดตามพื้นที่รื้อรอยจากภาพใบหน้าเพื่อให้ความรู้และติดตามส่วนบุคคล
ต่อมาเพิ่มบัญชีผู้ใช้ บันทึกสุขภาพ ประวัติและค่าประมาณรายบุคคล รายงานนี้สรุป Progress 1 ถึง
Progress 3 โดยแยกผลการพัฒนา หลักฐานทดสอบและแผนการทดลอง
ผลภาพและค่าพยากรณ์ไม่ใช้การวินิจฉัยหรือการรักษา

1 2 วัตถุประสงค์

ตรวจสอบคุณภาพภาพ ระบุตำแหน่งรื้อรอย และแสดงผลเชิงทดลองพร้อมข้อจำกัด
เชื่อมผลภาพกับแบบสอบถามและข้อมูลสุขภาพเพื่อช่วยติดตามปัจจัยที่อาจเกี่ยวข้อง
จัดหมวดคำแนะนำผลิตภัณฑ์ภายใต้กฎความปลอดภัย
บันทึกประวัติและแนวโน้ม โดยคำนึงถึงความยินยอมและสิทธิของเจ้าของข้อมูล

1 3 ขอบเขตโครงการ

ระบบรองรับบัญชีผู้ใช้และโปรไฟล์ การวิเคราะห์พื้นที่รื้อรอยจากภาพ การบันทึกเวลานอน
น้ำดื่มและข้อมูลรายวัน ประวัติย้อนหลัง 7 วัน และแนวโน้มหนึ่งวันถัดไปจากข้อมูลของบัญชี
นอกจากนี้ยังมีปฏิทินรอบเดือนและคำแนะนำผลิตภัณฑ์ตามข้อมูลที่ได้รับคามยินยอม

การทำนายอายุ การระบุตัวบุคคล การวินิจฉัยโรค
และการนำภาพผู้ใช้ไปฝึกโมเดลโดยอัตโนมัติอยู่นอกขอบเขตปัจจุบัน

1 4 พัฒนาการของโครงการ

Progress 1 กำหนดโจทย์ ศึกษาข้อมูลและสิทธิ์การใช้งาน วางสถาปัตยกรรม
และตัดการทำนายอายุออกจากขอบเขต

Progress 2 เชื่อมเว็บ API คิวงานและโมเดลรื้อรอย พร้อมสถานะงาน ภาพหน้ากากและภาพซ้อนสี
โดยระบุข้อจำกัดของผลทดลอง

Progress 3 เพิ่มบัญชี บันทึกสุขภาพ ประวัติ แนวโน้มและผลภาพรายบริเวณ
ข้อมูลผูกกับเจ้าของบัญชีและแยกค่าจริงจากค่าประมาณ

บทที่ 2 ทฤษฎีและเครื่องมือที่เกี่ยวข้อง

2 1 การแบ่งบริเวณรีร้อยจากภาพ

ระบบใช้โมเดล FFHQ Wrinkle แบบ UNet เพื่อแบ่งบริเวณที่มีลักษณะรีร้อย
ผลเป็นหน้ากากปกิเซลหรือ mask และภาพซ้อนสีหรือ overlay
สัดส่วนพื้นที่ที่ทำเครื่องหมายแสดงพื้นที่ที่โมเดลตรวจพบ
ไม่ใช่คะแนนสุขภาพผิวหรือระดับความรุนแรงทางคลินิก [1]

2 2 จุดตำแหน่งใบหน้าและขอบเขตรายบริเวณ

จุดตำแหน่งใบหน้าหรือ landmarks ใช้กำหนดพื้นที่สนใจ Region of Interest หรือ ROI
และสร้างภาพเส้นขอบแบบเรียบง่ายตามรูปหน้าของภาพอัปโหลด
การคำนวณรายบริเวณและการแสดงพื้นที่ต้องอ้างอิงขอบเขตเดียวกัน ภาพเส้นขอบเป็นภาพสรุปตำแหน่ง
ไม่ใช่หน้ากากปกิเซลจริง [2]

2 3 การพยากรณ์ส่วนบุคคล

Personal Forecast
ใช้แนวโน้มเชิงเส้นด้วยวิธีกำลังสองน้อยที่สุดจากเวลนอนและน้ำดื่มที่ผู้ใช้งานใน 7
วันของบัญชีเดียวกัน ต้องมีอย่างน้อย 3 ค่าต่อตัวแปรเพื่อประมาณหนึ่งวันถัดไป
วิธีนี้ต่างจากโมเดลพยากรณ์ผลสุขภาพที่ฝึกจากหลายบัญชี
และไม่ใช้ข้อมูลจำลองหรือข้อมูลบัญชีอื่นในเส้นทางส่วนบุคคล [3]

2 4 เครื่องมือในระบบ

เครื่องมือ	หน้าที่
Next.js และ FastAPI	หน้าเว็บ ตัวกลางฝั่ง server และ API ตรวจสอบคำขอ สิทธิและความยินยอม
PostgreSQL	เก็บบัญชี บันทึก สถานะงานและผลวิเคราะห์
Redis และ ARQ	จัดคิวให้ worker ประมวลผลแยกจากคำขอหน้าเว็บ
MinIO	เก็บภาพและไฟล์ผลวิเคราะห์แบบส่วนตัว

เครื่องมือ	หน้าที่
Label Studio	รองรับมนุษย์ทบทวนป้ายกำกับภาพตามความยินยอม
MLflow และ Trainer worker	ติดตามการทดลองและฝึกโมเดลรุ่นใหม่เพื่อรอตรวจรับ
Observation / observability	แยก actual reports จาก predictions; ตรวจ API, UV freshness, worker heartbeat, container health และ logs

2 4 1 Redis

คืออะไร: Redis เป็นระบบเก็บข้อมูลแบบ key-value ในหน่วยความจำ ใช้ส่งต่อข้อมูลขนาดเล็กและตอบสนองรวดเร็ว จึงเหมาะกับข้อมูลคิวและสถานะชั่วคราว ไม่ใช่ฐานข้อมูลหลักสำหรับประวัติสุขภาพหรือผลวิเคราะห์ [4]

โครงสร้าง: โครงการรัน Redis 7 ใน container แยก มีการยืนยันตัวตนด้วยรหัสผ่าน เปิด Append Only File (AOF) และเก็บข้อมูลใน volume ชื่อ redis_data ส่วนชื่อคิว inference และ training เป็นคิวงานที่ ARQ จัดการโดยใช้ Redis เป็นที่เก็บเบื้องหลัง

ทำงานโดยทั่วไป: แอปหรือผู้ส่งงานเขียนข้อความงานลงคิว แล้ว worker อ่านข้อความเมื่อพร้อมทำงาน Redis เก็บและส่งต่อข้อมูลคิว แต่ไม่ได้ประมวลผลฟังก์ชันของงานเอง ระบบที่ใช้ Redis จึงต้องมี worker และการติดตามผลแยกต่างหาก

นำมาใช้ในโครงการนี้: FastAPI ส่งรหัส analysis หรือข้อมูลอ้างอิงงานเข้า Redis โดยไม่แนบไฟล์ภาพทั้งไฟล์ งาน inference และ training แยกคิวกัน หากงานเสร็จหรือผิดพลาด สถานะถาวรและผลลัพธ์จะอ่านจาก PostgreSQL ผ่าน API; AOF ช่วยเก็บคิวข้ามการเริ่ม container แต่ไม่ได้รับรองว่างานจะสำเร็จ

รายละเอียดการทำงาน: ขอบเขตและการดูแล: Redis ช่วยลดการรอของคำขอเว็บ แต่ไม่ใช่แหล่งข้อมูลหลักของบัญชีหรือผลวิเคราะห์ หาก Redis หยุดทำงาน การส่งงานใหม่และการประสาน worker อาจสะดุด ขณะที่ประวัติที่บันทึกใน PostgreSQL ยังคงเป็นแหล่งอ่านผลถาวร AOF ช่วยคงข้อมูลบางส่วนข้ามการเริ่มบริการ แต่ไม่ยืนยันว่า job ทำเสร็จแล้ว ผู้ดูแลจึงต้องตรวจทั้งสถานะในฐานข้อมูลและ heartbeat ของ worker ไม่ใช่ดูเพียงคิวว่ามีข้อความหรือไม่

2 4 2 ARQ

คืออะไร: ARQ เป็นไลบรารีคิวงานแบบ asynchronous สำหรับ Python ที่ทำงานกับ Redis และ asyncio ใช้เลื่อนงานที่ใช้เวลานานออกจากคำขอเว็บ [5]

โครงสร้าง: ผู้ส่งงานเรียก enqueue_job พร้อมชื่อฟังก์ชันและข้อมูลที่จำเป็น ส่วน WorkerSettings ระบุฟังก์ชันที่อนุญาตให้ worker เรียก คิวที่ใช้งาน และพฤติกรรมของ worker เช่น จำนวนงานพร้อมกัน

ทำงานโดยทั่วไป: ARQ บันทึกงานใน Redis แล้ว worker ดึงงาน เรียกฟังก์ชันที่ลงทะเบียนไว้ และเก็บสถานะ/ผลตามกลไกของคิว งานที่อยู่ในสถานะ queued หมายถึงรอการทำงาน ไม่ได้แปลว่าทำเสร็จแล้ว

นำมาใช้ในโครงการนี้: FastAPI ส่ง run_inference พร้อม analysis_id เข้าคิว inference และส่งงานฝึกเข้าคิว training; มีงานเบื้องหลังสำหรับ cleanup และการส่งงานทบทวนภาพด้วย Label Studio เว็บอ่านสถานะผ่าน API เพื่อแสดง queued, running, completed หรือ failed

รายละเอียดการทำงาน: วงจรงานและข้อควรระวัง: โปรเจกต์แยกคิว inference กับ training และลงทะเบียนชื่อฟังก์ชันที่ worker อนุญาตให้เรียก ทำให้เว็บส่งงานแบบสั้นและคืนสถานะได้โดยไม่รอการคำนวณภาพหรืองานฝึก การตั้งเวลา readiness ของ Daily Health เป็น cron ที่ worker ตรวจสอบทุกวันจันทร์ 09:00 น. เวลาไทย แต่จะสร้าง candidate เฉพาะเมื่อข้อมูลจริงที่ได้รับ consent ผ่านเกณฑ์ การตั้งเวลาไม่ใช่หลักฐานว่ามีการฝึกหรือมีรุ่นใหม่ทุกสัปดาห์

2 4 3 Inference worker

คืออะไร: Inference worker

คือกระบวนการเบื้องหลังที่อ่านคิวและคำนวณผลจากโมเดลกับภาพที่ผู้ใช้อัปโหลด แยกจาก API เพื่อไม่ให้คำขอเว็บต้องรอการประมวลผลภาพจนเสร็จ

โครงสร้าง: worker ใช้ ARQ WorkerSettings เชื่อม Redis, PostgreSQL และ MinIO พร้อมเข้าถึงโค้ดวิเคราะห์ภาพและไฟล์โมเดล FFHQ-Wrinkle ที่ mount ให้ container ใช้งาน การตั้งค่าปัจจุบันจำกัด max_jobs เป็น 1

ทำงานโดยทั่วไป: worker รับรหัสงาน ตรวจสอบ metadata และไฟล์ต้นฉบับ ทำ validation และ inference จากนั้นบันทึกผลกับสถานะลงฐานข้อมูล และเก็บไฟล์ผลลัพธ์ใน object storage หากขั้นตอนใดล้มเหลวต้องบันทึกข้อผิดพลาดและกำหนดการจัดการไฟล์ให้เหมาะสม

นำมาใช้ในโครงการนี้: worker อ่าน analysis จาก PostgreSQL และภาพจาก MinIO ทำ quality checks, alignment, wrinkle/ROI analysis แล้วบันทึกผลเชิงโครงสร้างกลับ PostgreSQL ส่วน mask, overlay, regions และ outline เก็บใน MinIO; API ตรวจเจ้าของบัญชีและอายุไฟล์ก่อนส่ง artifact ให้นำเว็บ

รายละเอียดการทำงาน: ลำดับผลลัพธ์: job ใน Redis มี analysis ID ไม่แนบเนื้อหาภาพ worker อ่านสถานะและ object key จาก PostgreSQL แล้วดึงไฟล์จาก MinIO เปลี่ยนสถานะ queued เป็น running ก่อนเรียกบริการวิเคราะห์ที่ cache โมเดลไว้ใน process ผล JSON และสถานะกลับ PostgreSQL ส่วน mask, regions, overlay และ outline เก็บเป็น artifacts ใน MinIO พร้อมกำหนดอายุแสดงผล 24 ชั่วโมง การตั้ง max_jobs=1 จำกัด concurrency ของ worker หนึ่งตัว ไม่ใช่ข้อรับรอง throughput เมื่อโหลดสูง

2 4 4 Trainer worker

คืออะไร: Trainer worker เป็นกระบวนการเบื้องหลังสำหรับตรวจเงื่อนไขข้อมูล ผี candidate model และบันทึกผลประเมิน โดยแยกจาก inference worker ที่ให้บริการวิเคราะห์

โครงสร้าง: worker ใช้ ARQ กับคิว training มีบริการ/ฟังก์ชันผีที่กำหนดในโค้ด เชื่อมข้อมูลที่ได้รับอนุญาตกับพื้นที่เก็บ artifact และ MLflow สำหรับบันทึกการทดลอง การสร้าง candidate แยกจากการเลือกโมเดล active

ทำงานโดยทั่วไป: เมื่อถึงรอบตรวจ worker ประเมินจำนวนและคุณภาพข้อมูล จากนั้นส่งงานผีเมื่อผ่าน readiness gate เปรียบเทียบ candidate กับ baseline และชุดตรวจสอบ หากผ่านเกณฑ์ ผู้ดูแลยังต้องตรวจและอนุมัติก่อนเปิดใช้งาน

นำมาใช้ในโครงการนี้: งาน Daily Health ตรวจความพร้อมทุกวันจันทร์ 09:00 น. เวลาไทย และจับคู่ input วันที่ D กับ outcome จริงวันที่ D+1 ภายใต้ consent สำหรับ training; โมเดลสร้างเป็น candidate และต้องผ่านการประเมิน/อนุมัติ ส่วน generic time-series และ tabular trainer ที่มีอยู่ยังบันทึก metadata ไม่ใช่โมเดลที่ฝึกใช้งานจริง

รายละเอียดการทำงาน: สถานะจริงของแต่ละเส้นทาง: image-segmentation run ที่มี dataset ซึ่งผ่านการอนุมัติเรียกฟังก์ชันฝึกจริงและบันทึก run ใน MLflow ก่อนรอผู้ดูแลอนุมัติ ส่วน model family อื่นบางชนิดยังสร้างเพียง metadata-only run ที่ระบุ training_executed=false จึงไม่ควรรายงานว่าเป็นโมเดลที่ฝึกแล้ว Daily Health เป็นอีก pipeline หนึ่ง: ใช้ outcome ที่ผู้ให้รายงานเองและ consent ตรวจ readiness รายสัปดาห์ สร้าง candidate เมื่อข้อมูลพอ และไม่ deploy โดยอัตโนมัติ

2 4 5 PostgreSQL และ SQLAlchemy

คืออะไร: PostgreSQL เป็นฐานข้อมูลเชิงสัมพันธ์สำหรับข้อมูลที่ต้องคงอยู่และเชื่อมโยงกัน ส่วน SQLAlchemy เป็นไลบรารี Python ที่ช่วยให้โค้ดนิยามตารางและอ่าน/เขียนฐานข้อมูลได้เป็นระบบ [6], [7]

โครงสร้าง: บริการ PostgreSQL ใน Compose เก็บฐานข้อมูลแอป Aphrodize และฐานข้อมูล MLflow แยกกัน ตารางของแอปเชื่อมกับ consent, daily entries, outcomes, analyses, annotation tasks และ model registry ด้วยคีย์สัมพันธ์; SQLAlchemy ใช้ models, sessions และ transactions

ทำงานโดยทั่วไป: แอปเปิด session เชื่อมต่อฐานข้อมูล ใช้คำสั่งอ่านหรือเปลี่ยนแปลงข้อมูลภายใน transaction แล้ว commit หรือ rollback ตามผล การเปลี่ยน schema จัดการด้วย migration เพื่อให้โครงสร้างฐานข้อมูลสอดคล้องกับเวอร์ชันโค้ด

นำมาใช้ในโครงการนี้: FastAPI ตรวจ session ผู้ใช้ สิทธิเจ้าของข้อมูลและ consent แล้วใช้ SQLAlchemy อ่าน/บันทึกข้อมูล ผู้ใช้จึงเห็นประวัติและผลเดิมหลัง refresh ได้ PostgreSQL เก็บ metadata และผล JSON ส่วนไฟล์ภาพขนาดใหญ่เก็บใน MinIO

รายละเอียดการทำงาน: การแบ่งข้อมูลและความสอดคล้อง: Compose ใช้ PostgreSQL service เดียวแต่แยกฐานข้อมูลแอปกับฐานข้อมูล metadata ของ MLflow ออกจากกัน ผังแอปใช้ relational keys และข้อกำหนดของ schema เพื่อผูกกับผู้ใช้ เจ้าของข้อมูล consent สถานะ analysis training run และผลที่อ่านย้อนหลังได้ SQLAlchemy session ครอบการอ่าน/แก้ไขเป็น transaction เพื่อให้ commit หรือ rollback สอดคล้องกัน ข้อมูลไฟล์ภาพไม่ได้เก็บเป็น binary ในตาราง และการสำรองฐานข้อมูลอย่างเดียวจึงไม่ครอบคลุม MinIO

2 4 6 MinIO และ object storage

คืออะไร: MinIO เป็น object storage ที่รองรับ API แบบ S3 ใช้จัดเก็บไฟล์ binary เช่น ภาพและไฟล์ประกอบโมเดล โดยให้ข้อมูลไฟล์แยกจากตารางฐานข้อมูล [8]

โครงสร้าง: Compose รัน MinIO พร้อม data volume และสร้าง buckets สำหรับไฟล์ส่วนตัวของผู้ใช้งาน annotation และ MLflow artifacts; PostgreSQL เก็บ key/metadata ที่ใช้อ้างอิง object ไม่เก็บเนื้อหาภาพทั้งหมด

ทำงานโดยทั่วไป: แอปอัปโหลด object ไปยัง bucket ด้วย key แล้วบันทึกข้อมูลอ้างอิงไว้ในฐานข้อมูล เมื่อต้องอ่าน แอปตรวจสอบสิทธิ์และอายุการเข้าถึงก่อนส่งไฟล์หรือ URL ที่ควบคุมได้ การกำหนดอายุและลบ object ต้องมีงานดูแลแยกจากการลบแถวฐานข้อมูล

นำมาใช้ในโครงการนี้: ภาพต้นฉบับและผลประกอบการวิเคราะห์ถูกจัดเก็บใน MinIO โดย API ตรวจเจ้าของบัญชีก่อนส่งไฟล์; artifact แสดงผลมีเวลาหมดอายุ 24 ชั่วโมง ขณะที่ result JSON มีวงจรชีวิตแยกกัน ไฟล์โมเดล Daily Health และข้อมูล/โมเดล UV บางส่วนอยู่ใน local mounted directories

รายละเอียดการทำงาน: วงจรไฟล์: งานเริ่มจาก object key ที่ผูกกับผู้ใช้และ analysis ID ไม่เปิด bucket ให้ browser อ่านโดยตรง API ตรวจเจ้าของและเวลาใช้งานก่อนส่ง artifact ส่วน MinIO-init สร้าง bucket สำหรับไฟล์แอป annotation และ MLflow เมื่อเริ่ม stack การหมดอายุของภาพประกอบและการลบ metadata เป็นคนละงาน จึงต้องติดตามให้สอดคล้องกัน รวมทั้งทดสอบกู้คืนไฟล์และฐานข้อมูลแยกกันก่อนย้ายไป cloud

2 4 7 FastAPI

คืออะไร: FastAPI เป็น framework ภาษา Python สำหรับสร้าง HTTP API รับคำขอจากเว็บ ตรวจสอบข้อมูล และเรียกใช้บริการของระบบ [9]

โครงสร้าง: backend แบ่งเส้นทาง API, schemas, services, models และ dependencies สำหรับฐานข้อมูล การยืนยันตัวตนและการตั้งค่า เชื่อม PostgreSQL, Redis/ARQ และ MinIO ตามหน้าที่ของ endpoint

ทำงานโดยทั่วไป: API รับ request ตรวจสอบรูปแบบและตัวตน ตรวจสอบสิทธิ์/เงื่อนไขธุรกิจ แล้วอ่านหรือบันทึกข้อมูล หากเป็นงานยาวจะสร้าง job ให้ worker และคืนรหัสงานหรือสถานะเพื่อให้ client มาตรวจภายหลัง

นำมาใช้ในโครงการนี้: FastAPI ให้บริการสมัคร/เข้าสู่ระบบ บันทึก Daily Health ประวัติและผลวิเคราะห์ รับภาพ สร้างงาน inference และให้หน้าเว็บอ่านผลกับ artifacts; health endpoint ตรวจสอบความพร้อมพื้นฐาน แต่ไม่ได้ยืนยันว่าโมเดลหรือทุก integration ทำงานได้ครบ

รายละเอียดการทำงาน: ขอบเขตความเชื่อถือ: API เป็นจุดกลางสำหรับตรวจ schema, session, เจ้าของ record และ consent ก่อนอ่านหรือเปลี่ยนข้อมูล ไม่ควรให้ frontend เรียก PostgreSQL, Redis หรือ MinIO โดยตรง เส้นทาง analysis บันทึกกระเป๋าก่อนส่ง job จึงยังคืนผล/สถานะเดิมได้เมื่อ worker ทำงานช้า ส่วน endpoint health เป็นสัญญาณของบริการพื้นฐานและต้องเสริมด้วยการทดสอบเส้นทางจริงตั้งแต่รับ request จนอ่าน artifact

2 4 8 Next.js และส่วนติดต่อผู้ใช้

คืออะไร: Next.js เป็น framework React ที่ใช้สร้างหน้าเว็บและ server routes สำหรับรับส่งข้อมูลระหว่าง browser กับ backend [10]

โครงสร้าง: ส่วน frontend มีหน้าและ React components สำหรับแบบฟอร์ม dashboard, history, trends, profile และผลวิเคราะห์ รวมถึง API routes ฝั่ง server สำหรับ session และการส่งคำขอไป backend

ทำงานโดยทั่วไป: browser แสดงส่วนติดต่อและส่งข้อมูลไป server route; route ใช้ session ฝั่ง server เพื่อแนบข้อมูลยืนยันตัวตนที่จำเป็นและเรียก backend จากนั้นส่งเฉพาะผลที่หน้าเว็บต้องใช้กลับมา

นำมาใช้ในโครงการนี้: หน้าเว็บแสดงผลการวิเคราะห์ สถานะคิว บันทึกสุขภาพ กราฟและแผนที่ UV ผ่าน API ของ Aphrodize; session ใช้ cookie แบบ HttpOnly และ service/admin credentials อยู่ฝั่ง server ไม่ส่งตรงให้ JavaScript ใน browser

รายละเอียดการทำงาน: การปกป้อง credential: browser ใช้น้ำและ client components สำหรับฟอร์ม กราฟ และสถานะงาน ขณะที่ server routes ทำหน้าที่เป็น backend-for-frontend อ่าน session cookie แบบ HttpOnly แล้วเรียก Aphrodize API โดยไม่ส่ง service/admin secret เข้า JavaScript ฝั่ง client งานที่ใช้เวลานานจะแสดงสถานะจาก API และให้ผู้ใช้งานกลับมาอ่านผลผ่าน analysis ID ไม่เปิดการเชื่อมต่อด้านข้อมูลหรือ bucket จากหน้าเว็บ

2 4 9 MLflow

คืออะไร: MLflow เป็นระบบติดตามการทดลอง machine learning ใช้จัดเก็บ parameters, metrics, tags, run และการอ้างอิง artifacts เพื่อเปรียบเทียบการทดลองและตรวจย้อนกลับ [11]

โครงสร้าง: ใน Compose มี MLflow tracking server; backend store ใช้ฐาน PostgreSQL ชื่อ mlflow และ artifact store ที่ไป bucket mlflow ใน MinIO โดยใช้ S3-compatible endpoint และ credentials จาก environment

ทำงานโดยทั่วไป: ตัวฝึกเริ่ม run บันทึกพารามิเตอร์ ตัวชี้วัดและ artifact แล้ว MLflow แสดงและค้นหาข้อมูลเหล่านั้นภายหลัง การบันทึก run เป็นการติดตามการทดลอง ไม่ได้ฝึกโมเดลหรืออนุมัติ deployment ให้เอง

นำมาใช้ในโครงการนี้: งานฝึกภาพและ UV บันทึกการทดลองตาม pipeline ของตน ส่วน Daily Health ส่งตัวชี้วัดรวมและจำนวนข้อมูลโดยไม่ส่งข้อมูลสุขภาพรายบุคคล; การเปิดโมเดลต้องผ่าน registry/ชั้นอนุมัติแยกจากสถานะ run ใน MLflow

รายละเอียดการทำงาน: การอ่านผล run อย่างถูกต้อง: run ควรผูก model family, dataset/version หรือ manifest, configuration, metrics และ artifact ที่จำเป็นเพื่อให้เปรียบเทียบย้อนหลังได้ แต่ run ที่บันทึกสำเร็จอย่างเดียวไม่ได้พิสูจน์ว่าฝึกโมเดลจริง ตัวอย่าง smoke หรือ metadata-only run ใช้ตรวจการเชื่อมต่อได้เท่านั้น Daily Health ส่งเฉพาะ aggregate metrics และจำนวนข้อมูลโดยไม่ส่งแถวรายบุคคล MLflow Model Registry เป็นความสามารถทั่วไปของแพลตฟอร์ม [17] แต่เส้นทางอนุมัติของ Aphrodize แยกอยู่ใน application registry และต้องมีการตรวจโดยผู้ดูแล

2 4 10 Label Studio

คืออะไร: Label Studio เป็นเครื่องมือสร้างหรือทบทวน annotation โดยมนุษย์ เช่น การวาด mask บริเวณบนภาพเพื่อใช้ตรวจสอบผลของระบบ [12]

โครงสร้าง: บริการ Label Studio แยกจาก backend Aphrodize มี project, task, storage ของตัวเอง และเชื่อมจากโค้ดผ่าน SDK ด้วย URL, API token และ project ID; ภาพที่ส่งเป็นข้อมูล inline ใน task จึงมีสำเนาในระบบ Label Studio

ทำงานโดยทั่วไป: ระบบต้นทางสร้าง task พร้อมข้อมูลภาพและป้ายกำกับที่ต้องการ ผู้ตรวจทำ annotation ในหน้า Label Studio แล้วข้อมูลทบทวนถูกเก็บใน instance เพื่อให้ผู้รับผิดชอบตรวจสอบต่อการสร้าง task อย่างเดียวไม่ได้แปลว่าผลผ่านการตรวจหรือได้รับอนุญาตให้ใช้ฝึก

นำมาใช้ในโครงการนี้: Aphrodize เปิดสาขา review เมื่อมี consent สำหรับ annotation แยกและมี token/project ที่ใช้งานได้ ผลวิเคราะห์ถูกบันทึกก่อนเรียก SDK เพื่อไม่ให้การเชื่อมต่อผิดพลาดทำให้ผลสูญหาย ปัจจุบันยังไม่มี bridge ที่รับ annotation กลับ ตรวจสอบ และสร้าง dataset version เพื่อฝึกอัตโนมัติ

รายละเอียดการทำงาน: ขอบเขต human-in-the-loop: task ที่ส่งไปทบทวนเป็นสำเนาข้อมูลใน instance แยก จึงต้องใช้ consent เฉพาะด้าน กำหนดสิทธิ์ และมีนโยบาย retention/ลบทั้งสองฝั่ง หลังผู้ตรวจทำ annotation สามารถ export ผลเป็น JSON หรือรูปแบบที่รองรับ [18] แต่การ export ไม่เท่ากับการรับรองคุณภาพ ในระบบปัจจุบันยังขาดขั้นเชื่อมผลกลับมาที่ Aphrodize เพื่อตรวจสอบ บันทึกผู้อนุมัติ, สร้าง dataset manifest/version และส่งฝึก จึงต้องรายงานจุดนี้เป็น bridge ที่ยังไม่ครบ

2 4 11 UV refresh และ UV training

คืออะไร: UV refresh คือกระบวนการสร้างค่าพยากรณ์และ snapshot ให้ API อ่าน ส่วน UV training คือกระบวนการฝึกและประเมิน candidate model ใหม่ ทั้งสองเป็นงานคนละช่วงของวงจรโมเดล [13]

โครงสร้าง: บริการ UV ใช้สคริปต์ refresh_uv_forecast.py และ uv_mlops.py พร้อมไฟล์เดอร์ข้อมูล โมเดล และ artifacts ที่ mount ไว้ เชื่อม MLflow สำหรับติดตามการทดลอง; ชุดบริการนี้ทำงานแยกจากคิว ARQ

ทำงานโดยทั่วไป: refresh อ่านข้อมูลและโมเดลที่เลือกใช้เพื่อสร้าง snapshot ล่าสุด ส่วน training เตรียมข้อมูล ฝึก/ประเมิน candidate และบันทึกผล การตรวจสอบผลและอนุมัติรุ่นเป็นขั้นตอนสำคัญก่อนให้ API แสดงพยากรณ์

นำมาใช้ในโครงการนี้: เมื่อเปิด Compose profile ที่เกี่ยวข้อง uv-refresh ทำงานซ้ำทุก 6 ชั่วโมงหลังสำเร็จหรือ 30 นาทีหลังผิดพลาด ส่วน uv-training เริ่ม pipeline เมื่อ service เริ่มแล้วเว้น 30 วันหลังสำเร็จหรือ 1 วันหลังผิดพลาด; snapshot ที่ API ใช้ต้องผ่าน freshness check และพื้นที่ทดลองปัจจุบันครอบคลุมเฉพาะเมืองที่กำหนด

รายละเอียดการทำงาน: แหล่งข้อมูลและการเฝ้าระวัง: pipeline ใช้ข้อมูล UV clear-sky จาก TEMIS ซึ่งคำนวณภายใต้เงื่อนไขท้องฟ้าเปิด ไม่ใช่การวัด UV ทุกสภาพเมฆที่ระดับผู้ใช้ [19] โมเดล SARIMAX ถูกประเมินแบบย้อนหลังตามเวลาเทียบ baseline ก่อนลงทะเลเบียน candidate; refresh สร้าง snapshot สำหรับ API แยกจากรอบ train ระบบ UV ยังเก็บ forecast ก่อนถึง target date แล้วเมื่อ observation จริงมาถึงจึงคำนวณ error/bias ตามเมือง horizon และ version หากข้อมูลจริงยังไม่พอให้สถานะ collecting แทนการสรุปว่าโมเดลผ่าน

2 4 12 Docker Compose และการติดตามสถานะ

คืออะไร: Docker Compose เป็นเครื่องมือกำหนดและเริ่มบริการหลายตัวร่วมกันจากไฟล์ YAML ส่วน health checks และ logs ช่วยตรวจสอบสภาพเบื้องต้นและหาข้อผิดพลาดระหว่างทำงาน [14]

โครงสร้าง: compose.yml ระบุ images/build contexts, networks, environment variables, volumes, dependencies, ports และ profiles เช่น ai, background, demo และ uv-training; ข้อมูลถาวรถูกเก็บใน named volumes หรือโฟลเดอร์ที่ mount

ทำงานโดยทั่วไป: Compose เริ่ม container ตาม dependency และ profile ที่เลือก health check เรียกตรวจสอบสัญญาณพื้นฐานของแต่ละบริการ ส่วน logs แสดงเหตุการณ์ของ container; การตรวจ healthy ของบริการหนึ่งไม่ใช้การทดสอบการทำงานครบทั้งระบบ

นำมาใช้ในโครงการนี้: Compose ใช้เชื่อม API, PostgreSQL, Redis, MinIO และ worker รวมถึงบริการ AI/MLflow/Label Studio ตาม profile; monitoring API กับ logs สรุปความพร้อมและข้อผิดพลาดเบื้องต้น ขณะที่การสำรองและกู้คืน volume ต้องจัดการแยกจากการ restart

รายละเอียดการทำงาน: การปฏิบัติการ: profiles แยกบริการ AI, background, demo และ UV training เพื่อไม่เริ่มทุกงานโดยไม่จำเป็น named volumes คงข้อมูลของ PostgreSQL, Redis, MinIO และ Label Studio ส่วน health check ตรวจสอบเฉพาะบริการและ log rotation จำกัดขนาด log ในเครื่อง การที่ dependency เป็น healthy บอกเพียงว่า probe ผ่าน ณ เวลานั้น ไม่ได้ยืนยัน API key, project permission, model artifact หรือการทำงาน end-to-end; restart container ก็ไม่เท่ากับ backup หรือ recovery test

2 4 13 Observation tool และ Observability

Observation tool ของ Aphrodize ไม่ได้เป็น service แยก แต่เป็นส่วนบันทึก Daily Health และ self-report outcome ที่รับค่าจริงจากผู้ใช้และเก็บลง PostgreSQL โดยผูกกับบัญชีและวันที่ ใช้แสดง actual history และเป็น label สำหรับประเมินหรือฝึกได้เมื่อมี consent ตามขอบเขตข้อมูล

คำว่า observation ในบทนี้มีสองความหมายที่ต้องแยกกัน: ข้อมูลจริงที่ผู้ใช้รายงานเอง เช่น เวลานอน น้ำดื่ม และผลลัพธ์วันถัดไป ซึ่งใช้แสดง actual history และเป็น label ที่อาจใช้ฝึกเมื่อมี consent; กับ observability ซึ่งหมายถึงสัญญาณที่ผู้ดูแลใช้ตรวจสอบสุขภาพและพฤติกรรมของบริการ การทำนายของระบบไม่ใช่ observation และห้มนำมาแทนผลที่ผู้ใช้รายงานจริง

องค์ประกอบที่มีอยู่ประกอบด้วย API /api/v1/monitoring/analyses ซึ่งสรุปจำนวนงาน อัตราล้มเหลว quality flags และ p95 latency; /api/v1/monitoring/uv ซึ่งรายงานความสด คุณภาพ และสถานะ pipeline; UV lifecycle monitor ที่จับคู่ forecast เดิมกับ observation ที่มาภายหลังเพื่อคำนวณ error และ bias; รวมถึง Docker health checks/logs และ ARQ worker heartbeat

โดยทั่วไป observability ควรเชื่อม metrics, logs และ traces ให้ผู้ดูแลสอบถามเหตุการณ์ข้ามบริการได้ [15] เครื่องมืออย่าง Prometheus สามารถ scrape metrics endpoint เพื่อเก็บอนุกรมเวลาและสร้าง alert [16] แต่เครื่องมือเหล่านี้เป็นแนวทางต่อยอด ไม่ใช่บริการที่ติดตั้งอยู่ใน Compose ปัจจุบัน

สำหรับ Aphrodize ข้อมูลสุขภาพจริงและผลลัพธ์ที่ผู้ใช้รายงานเก็บใน PostgreSQL ตามบัญชีและวันที่ ส่วนการ monitor UV เก็บ archive ของ forecast และค่าจริงที่ตามมาภายใน local artifacts; MLflow แสดง runs และผลทดลอง แต่ไม่แทนการเฝ้าระวังสุขภาพบริการแบบ real-time ขอบเขตปัจจุบันจึงเป็น monitoring ขั้นต้น ยังไม่มี centralized metrics store, alert delivery หรือ distributed tracing

2 4 14 ความสัมพันธ์ระหว่างองค์ประกอบ

คืออะไร: ส่วนนี้เชื่อมคำอธิบายรายการบริการให้เห็นว่า metadata, image bytes, job state, model artifact และ observation เดินทางผ่านองค์ประกอบใด ไม่ใช่บริการเดี่ยวที่ติดตั้งเพิ่ม

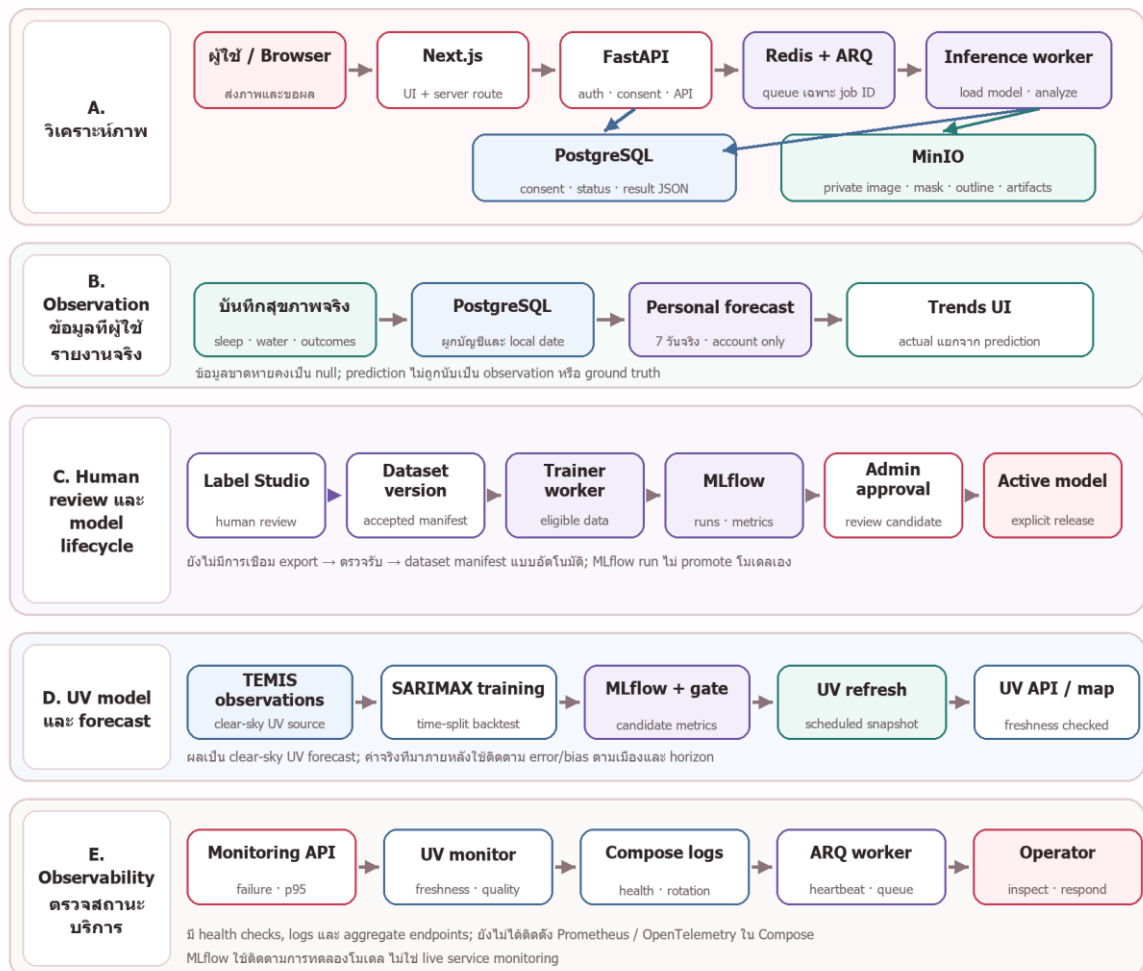
โครงสร้าง: เส้นทาง runtime ใช้ Browser/Next.js, FastAPI, PostgreSQL, MinIO, Redis/ARQ และ inference worker; เส้นทาง image review/training เพิ่ม Label Studio, dataset acceptance, trainer worker, MLflow และ application approval ส่วน Daily Health และ UV ใช้ observation/pipeline ของตน

ทำงานโดยทั่วไป: API ตรวจสอบตัวตน consent และสิทธิ์ก่อนบันทึกข้อมูลหรือสร้าง job worker อ่านเฉพาะ key/ID ที่จำเป็น ดึงไฟล์จาก object storage แล้วเขียนผลกลับฐานข้อมูลและ artifact store ผู้ใช้จึงอ่านผลผ่าน API ส่วนการฝึก การตรวจรับ annotation และการอนุมัติรุ่นแยกจาก inference

นำมาใช้ในโครงการนี้: รูปที่ 1 สรุปเส้นทางใช้งาน ภาพจริงที่ผู้ใช้รายงาน การฝึกโมเดล UV และจุดที่ผู้ดูแลตรวจระบบ เส้นประหมายถึงสะพานจาก Label Studio ไปยัง dataset version ที่ยังไม่ต่ออัตโนมัติ; run ใน MLflow ไม่ทำให้โมเดล active เอง

Aphrodize component, observation and model flow

ภาพรวมเส้นทางข้อมูลจริง การวิเคราะห์ การฝึก และการติดตามระบบ
 เส้นทึบ: เส้นทางที่มีในระบบ เส้นประ: ขั้นตอนที่ยังต้องตรวจรับ/ทำให้อัตโนมัติ



รูปที่ 1 แผนภาพการทำงานของบริการ ข้อมูล observation และวงจรโมเดล Aphrodize

2.5 สิทธิข้อมูลและความยินยอม

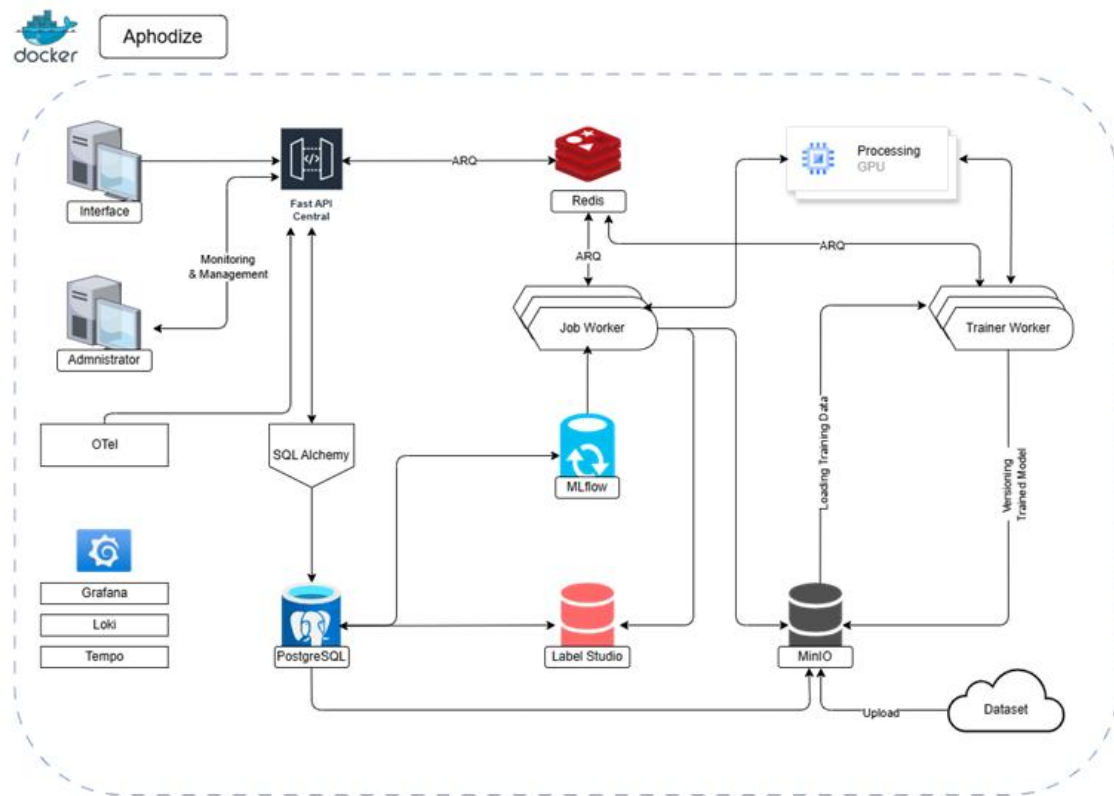
ชุดข้อมูลใดบ้างที่ต้องได้รับการตรวจสอบสิทธิการใช้งานและฐานกฎหมายที่เหมาะสม
 การเป็นชุดข้อมูลสาธารณะหรือมีสัญญาอนุญาตเพียงอย่างเดียวไม่ได้ยืนยันความยินยอมสำหรับการใช้งานทุกประเภท ระบบแยกวัตถุประสงค์การวิเคราะห์ การทบทวนโดยมนุษย์ และการฝึกโมเดล
 พร้อมตรวจสอบสิทธิเจ้าของบัญชี

บทที่ 3 การออกแบบและพัฒนาระบบ

3 1 สถาปัตยกรรมระบบ

ระบบเป็น modular monolith ร่วมกับ worker แยกกระบวนการ Next.js รับข้อมูลและแสดงผล ส่วน FastAPI ตรวจสอบคำขอและสิทธิ์ PostgreSQL เก็บข้อมูลเชิงโครงสร้าง MinIO เก็บไฟล์ภาพ และ Redis กับ ARQ ส่งงานให้ worker เพื่อไม่ให้งาน AI ขัดขวางการตอบคำขอหน้าเว็บ

Ecosystem Design



รูปที่ 2 สถาปัตยกรรมแนวคิดของ Aphrodize

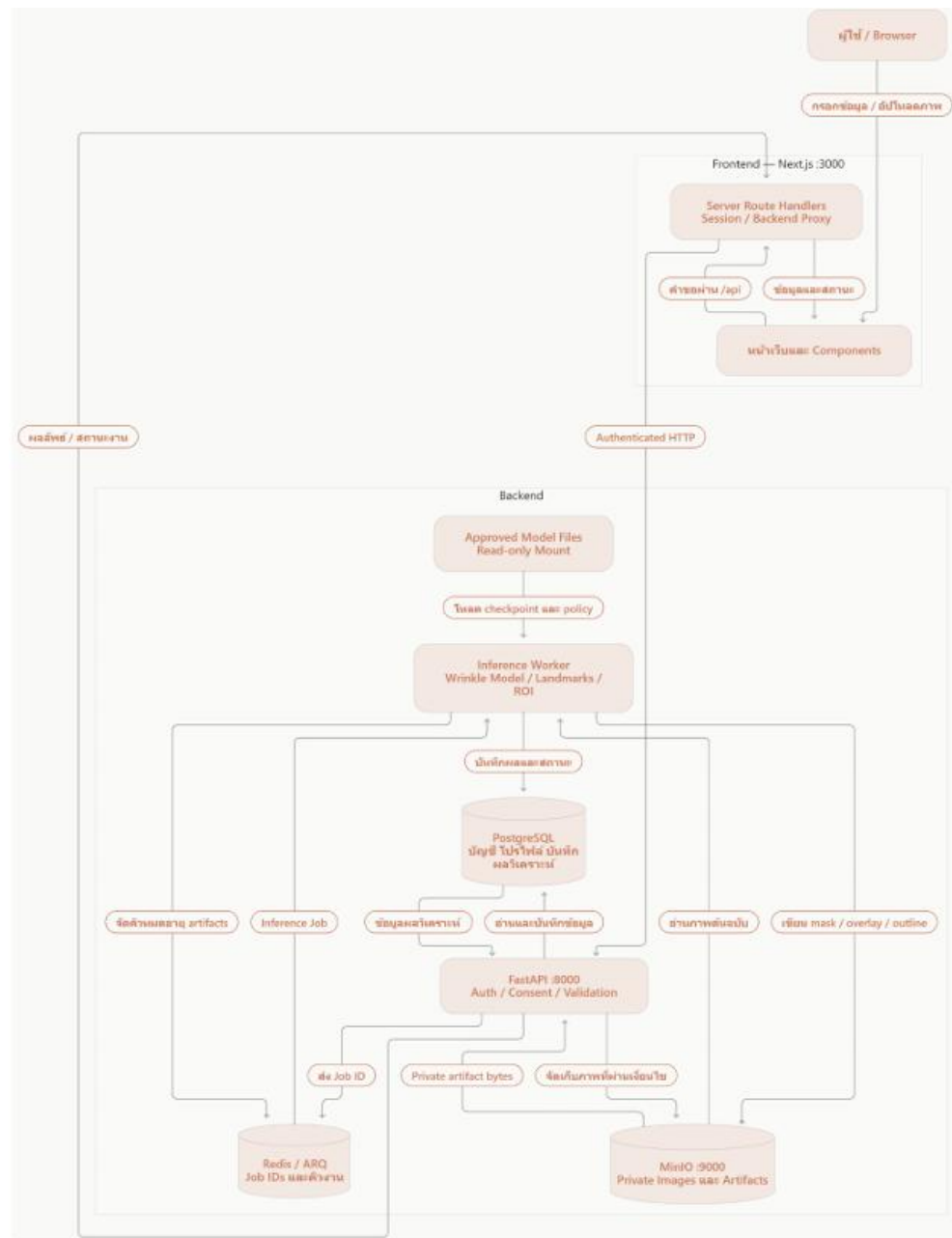
ภาพสถาปัตยกรรมเป็นแนวคิดที่รวมองค์ประกอบบางส่วนของแผนพัฒนา OpenTelemetry Grafana Loki และ Tempo ยังไม่ถือว่าติดตั้งครบตามต้นฉบับ ส่วน Label Studio ยังไม่มีการส่งผลตรวจรับไปสร้างชุดข้อมูลและฝึกใหม่อัตโนมัติ

3 2 เส้นทางการวิเคราะห์ภาพ

ผู้ใช้ให้ความยินยอมและส่งภาพ JPEG PNG หรือ WebP ไม่เกิน 10 MiB ผ่านเว็บ FastAPI ตรวจสอบชนิด ขนาด ความสมบูรณ์ และความละเอียดขั้นต่ำ 512 พิกเซลทางด้านสั้น

ภาพที่ผ่านถูกจัดเก็บและเข้าคิวเพื่อให้ inference worker ตรวจสอบภาพ
วิเคราะห์หรือรอยและสร้างผลประกอบ

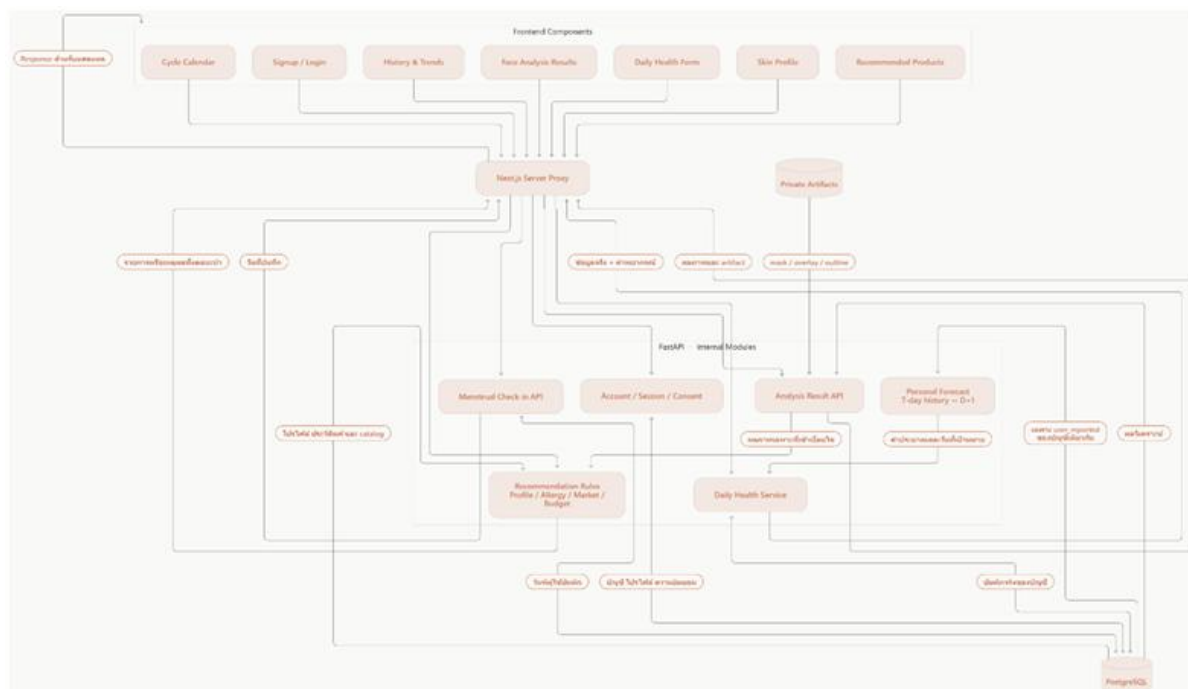
worker บันทึกสถานะและผลกลับฐานข้อมูล เว็บอ่านผลผ่าน API และแสดงสถานะกำลังทำงาน
ถูกปฏิเสธ ล้มเหลว หรือหมดอายุตามจริง ไฟล์ผลเป็นส่วนตัวและมีอายุการเข้าถึง 24 ชั่วโมง
ระบบมีขั้นตอนลบต้นฉบับหลังประมวลผล แต่ยังคงทดสอบการลบไฟล์และสิทธิ์เข้าถึงให้ครบทุกกรณี



รูปที่ 3 แผนภาพเส้นทางวิเคราะห์ใบหน้า

3 3 เส้นทางข้อมูลสุขภาพและการพยากรณ์

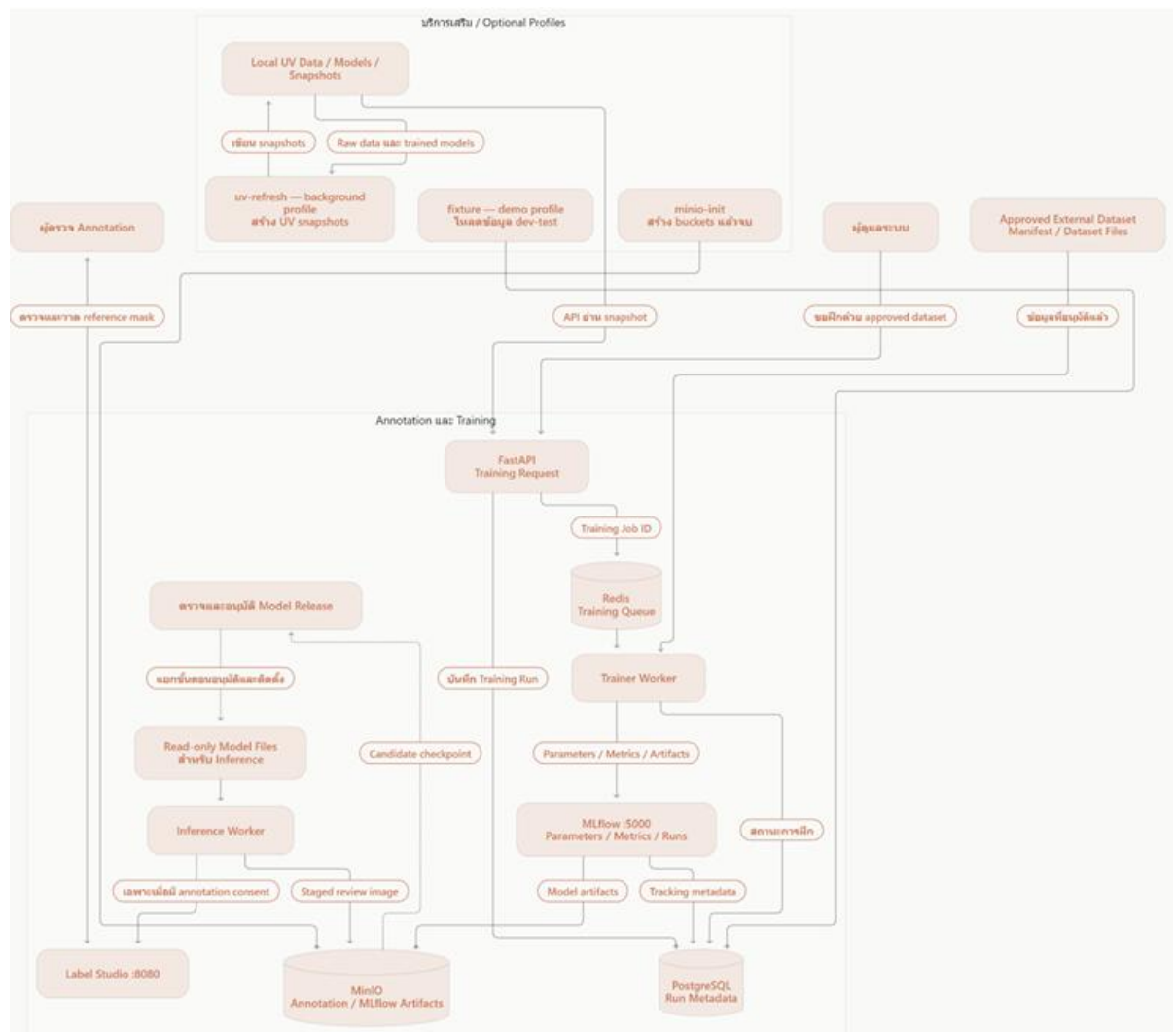
ผู้สมัครหรือเข้าสู่ระบบ ให้ความยินยอมและบันทึกข้อมูลรายวัน
ข้อมูลผู้กับเจ้าของบัญชีเพื่อแสดงประวัติและกราฟย้อนหลัง
การพยากรณ์ใช้เฉพาะข้อมูลจริงที่เข้าเงื่อนไขและมีความยินยอมแยก ไม่เต็มวันที่ขาดข้อมูลด้วยศูนย์



รูปที่ 4 แผนภาพการติดตามสุขภาพและการพยากรณ์

3 4 การทบทวนข้อมูลและพัฒนาโมเดล

Label Studio รองรับการทำงานป้ายกำกับภาพภายใต้ความยินยอมแยก ส่วน trainer worker
ฝึกโมเดลจากชุดข้อมูลที่ได้รับอนุญาตและผ่านการตรวจรับ MLflow บันทึกการทดลองและตัวชี้วัด
การสร้างงานทบทวนภาพไม่เท่ากับข้อมูลพร้อมฝึก และการฝึกเสร็จไม่เท่ากับอนุมัติให้เปิดใช้



รูปที่ 5 แนวคิดการทบทวนป้ายกำกับภาพ

3 5 แผนการทดลองและประเมิน

การประเมินต่อไปนี้เป็นแผน ไม่ใช่ผลความแม่นยำที่วัดแล้ว ต้องบันทึกเวอร์ชันข้อมูล โค้ด ไฟล์โมเดลและเงื่อนไขเพื่อให้ทำซ้ำได้

งานวิจัยใช้ภาพที่ได้รับอนุญาตและหน้ากากอ้างอิงจากผู้ประเมิน แยกชุดฝึก ชุดตรวจสอบและชุดทดสอบตามบุคคล ปรับเกณฑ์บนชุดตรวจสอบเท่านั้น แล้วเปรียบเทียบก่อนและหลังใช้ landmark ROI ด้วยไฟล์โมเดลและการเตรียมภาพเดียวกัน วัด Dice IoU precision recall ความคลาดเคลื่อนรายบริเวณ อัตราปฏิเสธภาพและเวลาประมวลผล

งานพยากรณ์ใช้การทดสอบเลื่อนจุดเริ่มต้นตามเวลา วัน D ใช้ข้อมูลที่มีถึงวัน D เพื่อประมาณวัน D+1 แล้วเทียบกับค่าที่ผู้ใช้รายงานจริง เปรียบเทียบกับค่าล่าสุดและค่าเฉลี่ยย้อนหลัง 7 วัน รายงาน MAE และ RMSE ของเวลานอนเป็นนาฬิกาและน้ำดื่มเป็นมิลลิลิตร พร้อมจำนวนบัญชีและสัดส่วนวันที่พยากรณ์ได้

การทดสอบเชื่อมระบบต้องครอบคลุมสมัคร เข้าสู่ระบบ ให้ความยินยอม บันทึก อ่านประวัติ และพยากรณ์ รวมถึงอัปโหลด เข้าคิว ประมวลผลและอ่านผล ตรวจสอบการแยกบัญชี วันที่ขาดข้อมูล งานล้มเหลวและไฟล์หมดอายุ โดยบันทึกผลผ่านหรือไม่ผ่าน รหัส HTTP และเวลารอโดยไม่เก็บรหัสลับ

บทที่ 4 ผลการพัฒนาและการทดสอบ

4 1 ส่วนติดต่อผู้ใช้และ Frontend service (Next.js)

ผลการพัฒนาเว็บครอบคลุมการสมัครและเข้าสู่ระบบ การจัดการ session ผัง server และหน้าหลักที่เชื่อมไปยังบริการของระบบ โปรไฟล์และข้อมูลรายวันแสดงตามบัญชีที่เข้าสู่ระบบ โดยหน้า Dashboard สรุปจำนวนวันที่มีข้อมูล ค่าเฉลี่ย และประวัติย้อนหลัง

หน้า Home ใช้ภาพประกอบเป็นส่วนหนึ่งของ hero และมีแถบนำทางไปยังหน้าจริง ส่วนหน้าข้อมูลจัดลำดับองค์ประกอบให้ดูข้อมูลหลักได้สะดวกและปรับการแสดงผลตามขนาดหน้าจอ

หน้า Trends แยกข้อมูลจริงด้วยเส้นทึบและค่าประมาณด้วยเส้นประ พร้อมป้ายวันที่และหน่วย เมื่อมีข้อมูลไม่พอจะแสดงสถานะตามจริง

หน้าเว็บไม่ได้สร้างจุดพยากรณ์หรือช่วงความเชื่อมั่นที่ไม่มีข้อมูลรองรับ

ปฏิทินรอบเดือนรองรับการบันทึกหรือแก้ไขวันที่ผ่านมาและวันที่ปัจจุบันภายใต้ความยินยอม ไม่รับวันอนาคตเป็นข้อมูลจริง การประมาณวันรอบถัดไปใช้ข้อมูลวันเริ่มรอบตามเงื่อนไขของโปรแกรม และอาจยังไม่แสดงเมื่อประวัติไม่เพียงพอ

หน้าคำแนะนำผลิตภัณฑ์ใช้โปรไฟล์ผิว ความไวต่อการระคายเคือง ประวัติแพ้ และเค็ดตาด็อกที่ผ่านการตรวจทาน ผู้ใช้กรอกลงและราคาสูงสุดได้ หากไม่มีรายการตรงเงื่อนไขจะแสดงสถานะตามจริง ภาพในรายงานเป็นหลักฐานหน้าจอ ไม่ใช่การรับรองว่าโมเดลแม่นยำหรือผลิตภัณฑ์จะเหมาะกับทุกคน

หลักฐานที่รายงาน ณ 6 ตุลาคม 2026 ระบุว่า Frontend lint และ TypeScript type check ผ่าน การจัดรูปแบบเอกสารครั้งนี้ไม่ได้รับคำสั่งเหล่านี้ซ้ำ

4 2 FastAPI service

FastAPI รับคำขอจากเว็บ ตรวจสอบรูปแบบข้อมูล ตัวตน สิทธิเจ้าของบัญชี และความยินยอมก่อนอ่านหรือบันทึกข้อมูล บริการครอบคลุมการยืนยันตัวตน ข้อมูลสุขภาพ การอัปเดตภาพ การสร้างงานวิเคราะห์ และการอ่านผลหรือไฟล์ประกอบ

งานที่ใช้เวลานานถูกส่งต่อไปยัง worker และ API คืนรหัสหรือสถานะให้หน้าเว็บตรวจภายหลัง การตรวจ health endpoint ยืนยันเพียงความพร้อมพื้นฐาน ไม่ได้ยืนยันว่าโมเดลหรือการเชื่อมต่อกับทุกบริการทำงานครบ

ชุดทดสอบที่รายงานรวมไฟล์ tests/test_api_auth.py ซึ่งตรวจกรณีเส้นทางสาธารณะ การปฏิเสธข้อมูลยืนยันตัวตนที่ไม่ถูกต้อง และการยอมรับข้อมูลยืนยันตัวตนที่ตั้งค่าไว้ ผลรายงานเป็นจำนวนรวมของทั้งคู่ ไม่ได้แยกจำนวนผ่านตาม service

4 3 PostgreSQL service และ SQLAlchemy

PostgreSQL เก็บข้อมูลที่ต้องคงอยู่และเชื่อมโยงตามบัญชี ได้แก่ session/consent ข้อมูลรายวัน ผลที่ผู้ใช้รายงาน งานวิเคราะห์ สถานะงาน ผลลัพธ์ และข้อมูลรุ่นโมเดล SQLAlchemy เป็นชั้นที่ backend ใช้กำหนด model, เปิด session และอ่านเขียนข้อมูล

ผลเชิงการทำงานคือหน้าเว็บสามารถขอประวัติและผลที่บันทึกไว้จาก API ได้อีกครั้งหลัง refresh โดยไม่ต้องอาศัยข้อมูลใน browser เพียงอย่างเดียว ในการตรวจระบบที่รายงานมี health check ของฐานข้อมูล แต่ไม่มีผลทดสอบ live integration ของ PostgreSQL แยกเป็นราย service

4 4 Redis และ ARQ queue service

Redis ใช้เก็บและส่งต่องานเบื้องหลัง ส่วน ARQ เป็นตัวจัดการคิวและเรียกฟังก์ชัน Python ใน worker โดยโครงการแยกคิว inference ออกจาก training รหัสงานถูกส่งในคิว ขณะที่ไฟล์ภาพอยู่ใน storage

ผลที่ผู้ใช้เห็นคือคำขอวิเคราะห์ไม่ต้องรอ worker ทำงานเสร็จใน HTTP request เดียว หน้าเว็บตรวจสอบสถานะงานผ่าน API ได้ การตรวจสอบสถานะ Redis/worker ที่รายงานเป็น health check และข้อมูล heartbeat ไม่ใช่หลักฐานว่าทุก job สำเร็จครบวงจร

4 5 MinIO object storage service

MinIO เก็บไฟล์ต้นฉบับและไฟล์ประกอบผลวิเคราะห์แยกจากข้อมูลตารางใน PostgreSQL API เป็นผู้ตรวจบัญชีเจ้าของและอายุการเข้าถึงก่อนส่งไฟล์ให้หน้าเว็บ ไฟล์แสดงผลมีอายุที่กำหนดไว้ ขณะที่ result JSON มีวงจรชีวิตแยกจากไฟล์ภาพ

ชุดทดสอบ tests/test_analysis_artifacts.py ที่อยู่ในรายงานครอบคลุมกติกาสีทธิและอายุไฟล์ในระดับโค้ด การตรวจดังกล่าวไม่ได้ยืนยันการทดสอบ upload/download กับ MinIO container จริงทุกกรณี

4 6 Inference worker และบริการวิเคราะห์ภาพ

ใน Progress 2 เส้นทางวิเคราะห์ภาพเชื่อมเว็บ, API, คิวงาน และโมเดล พร้อมแสดง mask และ overlay ขณะนั้นผลยังอิงการวิเคราะห์ล่าสุดของ browser

และนโยบายความเชื่อมั่นยังไม่ผ่านการปรับเทียบ จึงจัดเป็นผลทดลอง

ผลใน Progress 3 แสดงสัดส่วนพื้นที่ที่ทำเครื่องหมายและขอบเขตใบหน้าที่ได้จาก landmarks พร้อมสรุปเฉพาะ ROI ที่มีผลจาก API ไม่เติมค่าเรื่องรวมชน รอยแดง หรือความกระชับเมื่อไม่มีข้อมูลรองรับ สัดส่วนพื้นที่เป็นค่าที่โมเดลทำเครื่องหมาย ไม่ใช่คะแนนสุขภาพผิวหรือการวินิจฉัย

ชุดทดสอบที่รายงานรวม tests/test_quality_gate.py, tests/test_inference_worker.py, tests/test_analysis_artifacts.py, tests/test_landmark_rois.py และ tests/test_personal_outline.py ผลรวมของไฟล์ที่ระบุในตารางคือ 32 passed, 1 warning และ 0 failed การทดสอบเหล่านี้ตรวจตราระยะหลายจุด แต่รายงานระบุว่ายังไม่มีหลักฐานการทดสอบภาพแบบครบเส้นทางบนบริการทั้งหมด

4 7 Daily Health API และ personal forecast service

Daily Health รับบันทึกสุขภาพและผลที่ผู้ใช้รายงานจริงโดยผูกกับบัญชี ส่วน personal forecast ใช้ประวัติจริงของบัญชินั้นสำหรับตัวชี้วัดเวลานอนและน้ำดื่มในช่วงย้อนหลัง 7 วัน ต้องมีข้อมูลอย่างน้อย 3 รายการต่อ metric ก่อนคำนวณค่าประมาณวันถัดไป และวันที่ไม่มีข้อมูลไม่ถูกแทนด้วยศูนย์

กราฟใน Trends แสดงค่าที่บันทึกและค่าคาดการณ์แยกกัน พร้อมหน่วยและวันเป้าหมาย ผลทำนายไม่ใช่ผลที่ผู้ใช้รายงานจริง ชุดทดสอบ tests/test_daily_health_personal_forecast.py อยู่ในผลรวม 32 กรณีที่รายงาน โดยมีกรณีประวัติจริงของบัญชี การปฏิเสธเมื่อข้อมูลไม่พอ และเงื่อนไข consent

4 8 Trainer worker และ MLflow services

Trainer worker แยกงานฝึก candidate ออกจาก worker ที่ให้บริการ inference ส่วน MLflow บันทึกข้อมูลการทดลองและ metrics การสร้าง run หรือ candidate ไม่ได้ทำให้โมเดลถูกเปิดใช้งานโดยอัตโนมัติ การเปลี่ยนรุ่นยังต้องผ่านการตรวจและอนุมัติตามขั้นตอนของระบบ

หลักฐานที่สรุปในรายงานต้นฉบับระบุว่ายังไม่ยืนยัน trainer build ในรอบตรวจที่รายงาน generic time-series/tabular worker ยังไม่ควรนับเป็นโมเดลที่ผ่านการฝึกและประเมินใช้งานจริง จึงไม่มีผลความแม่นยำราย service ให้สรุปในบทนี้

4 9 Label Studio review service

Label Studio เป็นบริการแยกสำหรับผู้ตรวจสอบสร้างหรือทบทวน annotation ภาพ การส่งภาพเข้าทบทวนต้องมีความยินยอมสำหรับ annotation และการตั้งค่า URL, token และ project ที่ใช้งานได้ ผลวิเคราะห์หลักถูกบันทึกก่อนเริ่มเชื่อมต่อบริการนี้

รายงานระบุ health check ของหน้า login แต่ยังไม่มีความหลักฐานการทดสอบครบตั้งแต่ SDK สร้าง task, ผู้ตรวจสอบ annotation กลับ, ตรวจรับผล และสร้าง dataset version สำหรับ training ดังนั้นการมี task ไม่เท่ากับวงจร human review ที่เชื่อมถึงการฝึกเสร็จสมบูรณ์

4 10 UV forecast และ training services

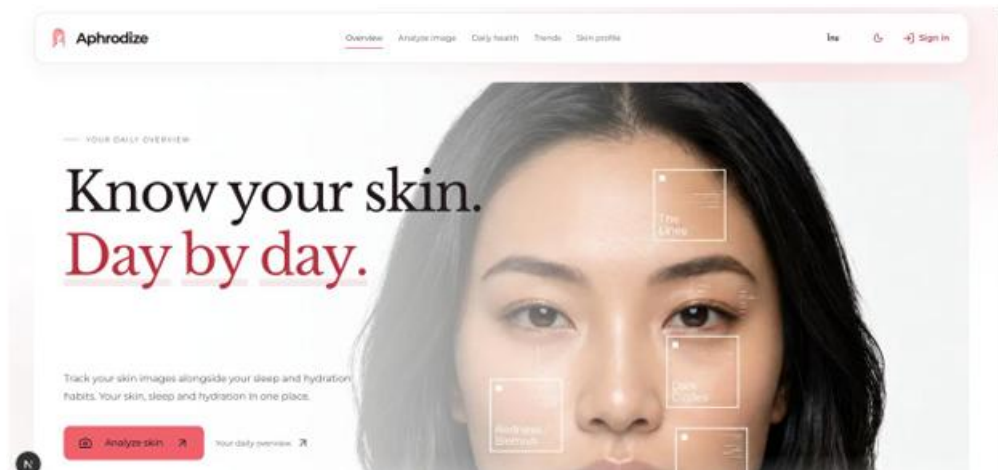
งาน UV refresh สร้าง snapshot ให้ API อ่าน ส่วน UV training สร้างและประเมิน candidate แยกจากการให้บริการ forecast บริการทั้งสองใช้ profile และไฟล์เดอร์ข้อมูล/โมเดลที่กำหนดไว้ รวมถึงส่งข้อมูลการทดลองไปยัง MLflow ตามการตั้งค่า

บันทึกตรวจระบบที่ใช้อ้างอิงในเอกสารระบุว่า UV services ยังไม่ได้เปิดในรอบ restart ล่าสุด และ snapshot ยังไม่พร้อม จึงยังสรุปผลการให้บริการ UV แบบครบเส้นทางไม่ได้ ต้องตรวจการเริ่มบริการ ความสดของ snapshot และการอนุมัติรุ่นก่อนใช้ผล

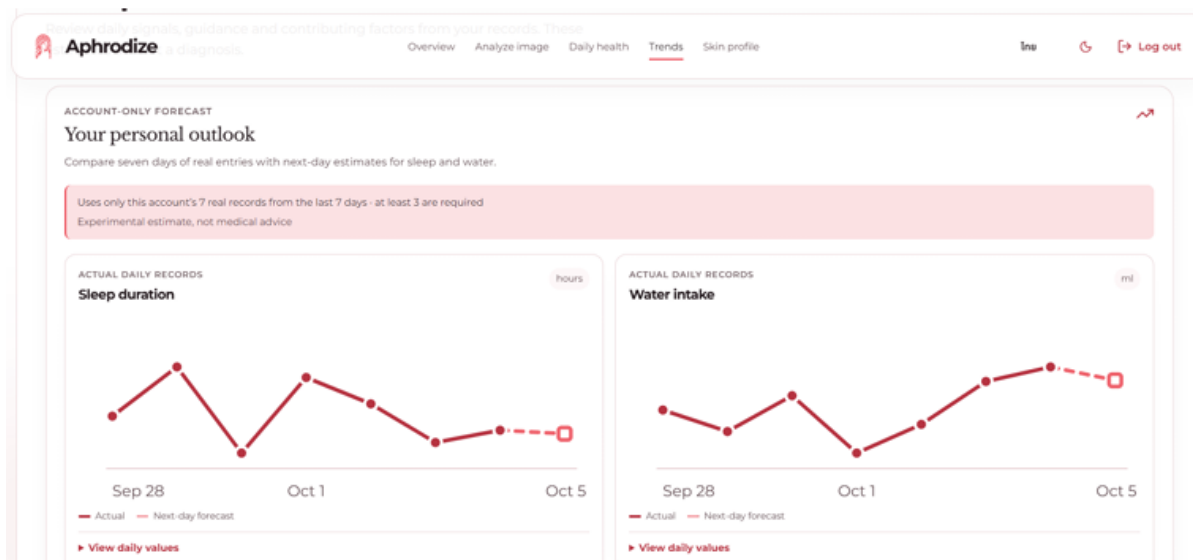
4 11 สรุปหลักฐานการทดสอบราย service

ผลทดสอบอัตโนมัติและ frontend ที่อ้างในบทนี้เป็นผลที่รายงานไว้ ณ 6 ตุลาคม 2026 ไม่ได้รันซ้ำระหว่างจัดทำเอกสารฉบับนี้ ตารางสรุปผลรวมทั้งชุดทดสอบ lint, type check และสถานะ health check ที่ต้นฉบับรายงาน

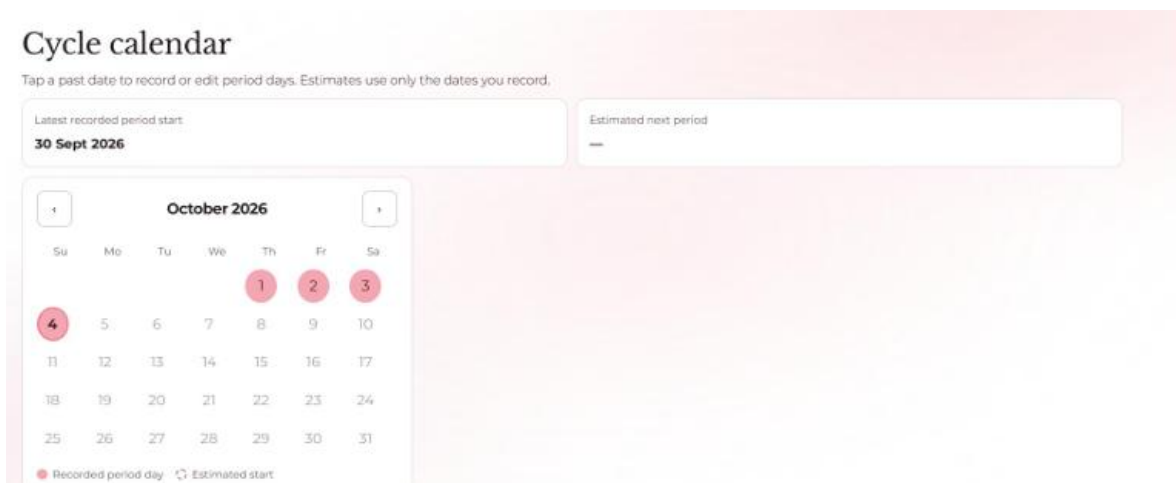
จำนวนผ่าน 32 กรณีเป็นผลรวมของไฟล์ทดสอบที่ระบุ ไม่ใช่จำนวนผ่านแยกตาม service; health check ก็ไม่ยืนยันว่าเว็บ API, Redis/ARQ, worker และ storage ผ่านการทำงานร่วมกันทุกเส้นทาง การตรวจรับระบบจริงจึงยังต้องทำ end-to-end โดยเฉพาะการวิเคราะห์ภาพ, Label Studio, trainer และ UV



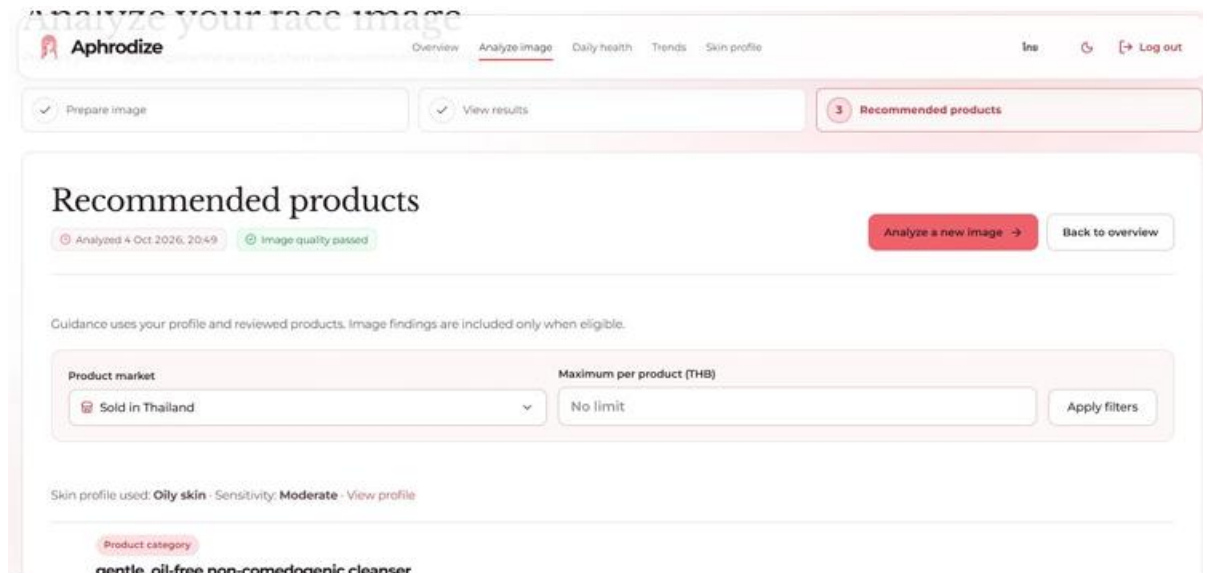
รูปที่ 6 หน้า Home



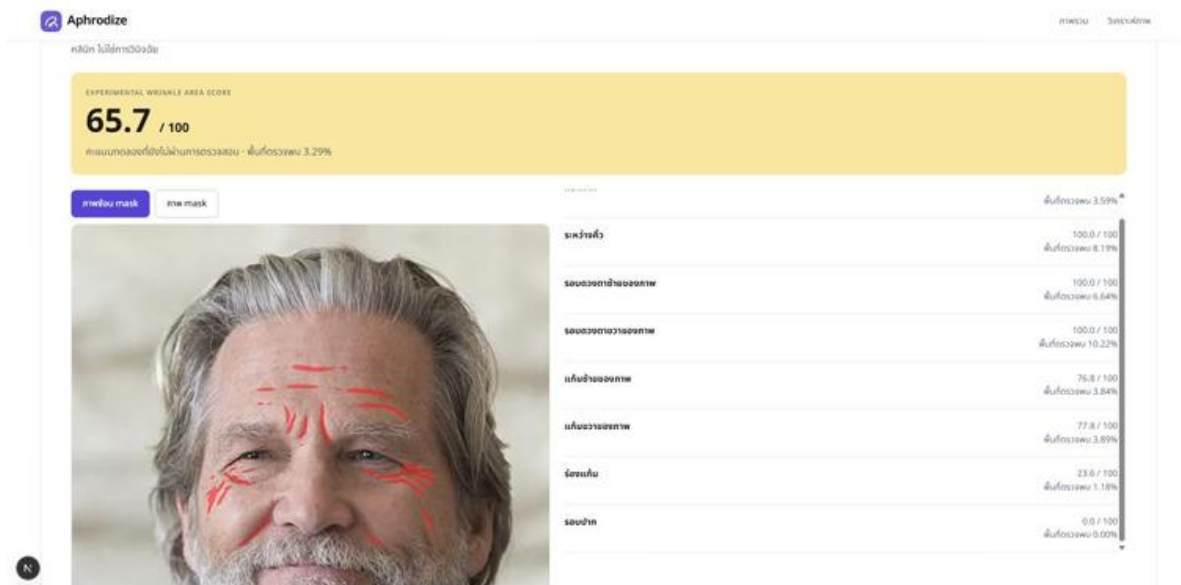
รูปที่ 7 หน้า Trends



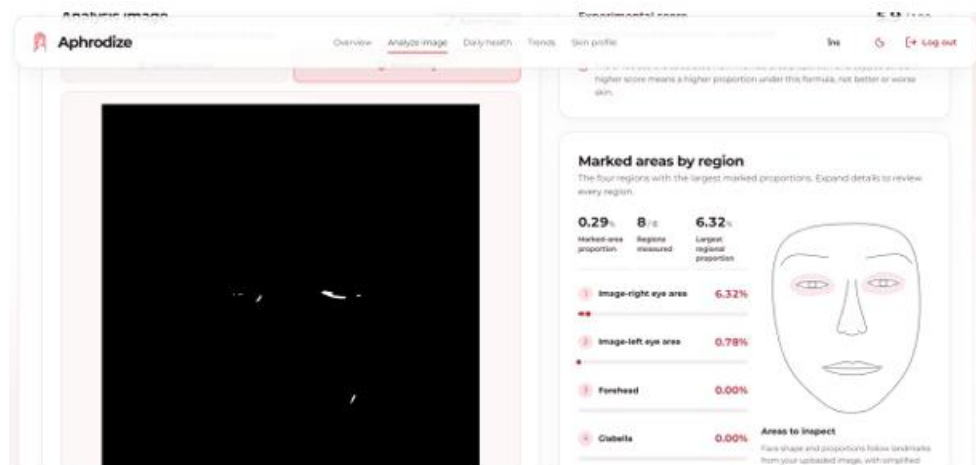
รูปที่ 8 หน้าปฏิทินรอบเดือน



รูปที่ 9 หน้าแนะนำผลิตภัณฑ์



รูปที่ 10 หน้าวิเคราะห์ภาพใน Progress 2



รูปที่ 11 หน้าผลวิเคราะห์ใบหน้าใน Progress 3

การตรวจสอบ	ผลที่ต้นฉบับรายงาน ณ 6 ตุลาคม 2026
ชุดทดสอบอัตโนมัติ	32 passed 0 failed 1 warning ใช้เวลา 15.70 วินาที
Frontend lint	ผ่าน ไม่พบ lint error
TypeScript type check	ผ่าน ไม่พบ TypeScript error
บริการและ worker	รายงาน health check ของบริการ แต่ยังไม่มีหลักฐานทดสอบภาพแบบครบเส้นทาง และยังไม่ยืนยัน trainer build

บทที่ 5 สรุปและข้อเสนอแนะ

5 1 สรุปผลการพัฒนา

โครงการพัฒนาจากการกำหนดโจทย์และสถาปัตยกรรมไปสู่ต้นแบบวิเคราะห์พื้นที่ที่รกรอ และต่อยอดเป็นระบบที่มีบัญชี ข้อมูลสุขภาพ ประวัติ กราฟและค่าประมาณรายบุคคล ผลภาพรวมบริเวณช่วยให้ตรวจตำแหน่งที่โมเดลทำเครื่องหมายได้ แต่ยังไม่ยืนยันความแม่นยำทางคลินิก

5 2 ข้อจำกัด

ยังไม่มีผล Dice IoU MAE หรือ RMSE ของกลุ่มผู้ใช้เป้าหมายที่ยืนยันในรายงาน และยังต้องจัดชุดตรวจสอบที่มีสิทธิ์และแยกจากชุดฝึกให้เพียงพอ การอนุมัตินโยบายโดยผู้ดูแลไม่เท่ากับการปรับเทียบทางสถิติหรือการรับรองทางคลินิก

วงจรถบภาพกับการฝึกใหม่ยังไม่เป็นอัตโนมัติครบขั้นตอน การมี landmarks หรือหน้าจอตีละเอียดขึ้นไม่ได้พิสูจน์คุณภาพโมเดล และยังต้องตรวจการแยกบัญชี สิทธิไฟล์ การหมดอายุและการลบข้อมูลแบบครบเส้นทาง

5 3 ข้อเสนอแนะและงานถัดไป

5 3 1 โครงสร้างระบบที่ต่อยอดได้

คง Next.js เป็นส่วนติดต่อผู้ใช้และ FastAPI เป็น API หลักในระยะแรก โดยออกแบบ API ให้ไม่เก็บ session state ไว้ในหน่วยความจำของ instance เพื่อเพิ่มหรือลดจำนวน instance ได้ ส่วนงาน inference, training และ UV แยกเป็น worker pool ตามภาระงาน ทำให้ปรับทรัพยากรและ deploy แต่ละกลุ่มได้อิสระ ควรเริ่มแยกเป็น microservice เพิ่มเมื่อมีเหตุผลด้านปริมาณงาน ทีมดูแล หรือวงจร release ที่ต่างกัน

พัฒนาการทำงานผ่าน Redis/ARQ ให้ทนต่อการหยุดชะงัก โดยกำหนด timeout, จำนวน retry, การป้องกันงานซ้ำ (idempotency) และวิธีตรวจ/กู้คืนงานที่ล้มเหลวให้ชัดเจน API ควรบันทึกภาระงานและสถานะลงฐานข้อมูล เพื่อให้ผู้ใช้กลับมาตรวจผลได้ แม้ worker หรือ container จะเริ่มใหม่

กำหนด PostgreSQL เป็นแหล่งข้อมูลหลักสำหรับบัญชี consent ประวัติและผลวิเคราะห์ ส่วนไฟล์ภาพและ artifacts เก็บใน object storage โดยบันทึก key, เจ้าของ, ชนิดข้อมูลและวันหมดอายุให้ตรวจสอบได้ ควรมี schema migration ที่ตรวจย้อนกลับได้ และแผนสำรอง/กู้คืนฐานข้อมูลกับไฟล์แยกกัน แยกวงจรฝึกและให้บริการโมเดลออกจากกัน: เก็บ candidate, ชุดข้อมูลและ provenance, metrics, ผู้อนุมัติ และรุ่นที่ active ในระบบ registry; ใช้ MLflow ติดตามการทดลอง แล้ว deploy เฉพาะ artifact ที่ผ่านเกณฑ์พร้อม manifest/checksum มีวิธีย้อนกลับรุ่นเดิม และสร้าง dataset version จาก annotation ที่ผ่านการตรวจรับและ consent แล้ว

5 3 2 แนวทางนำระบบขึ้น Cloud

เริ่มจากบริการ container แบบ managed ที่รองรับ Next.js/API และ worker แยกกัน แทนการดูแล Kubernetes เองตั้งแต่ระยะแรก เลือกผลิตภัณฑ์เทียบเท่าจาก cloud provider ที่องค์กรใช้ได้ โดยให้แต่ละ service ใช้ image ที่สร้างเวอร์ชันเดียวกันกับการพัฒนาในเครื่อง และเก็บ image ใน private container registry

ย้าย PostgreSQL ไป managed database ที่อยู่ใน private network เปิด automated backup และ point-in-time recovery; ใช้ managed Redis สำหรับคิว; และใช้ object storage ที่รองรับ S3 สำหรับ MinIO buckets พร้อม encryption, lifecycle/retention policy และ versioning ตามความจำเป็น หากยังต้องใช้ MinIO ให้รันเป็นบริการ container พร้อม persistent storage และแผนสำรองที่ทดสอบกู้คืนได้

ให้ browser เข้าถึงเว็บผ่าน HTTPS/CDN หรือ load balancer ส่วน API และ workers ติดต่อฐานข้อมูล คิว และ object storage ผ่านเครือข่ายภายใน ใช้ secret manager เก็บรหัสผ่านและ API keys, service identity แบบ least privilege, แยกสิทธิ์อ่าน/เขียนตาม bucket และห้ามฝัง secret ไว้ใน image, source code หรือ browser

แยก development, staging และ production ด้วย configuration, credentials, database และ storage ของตนเอง ตั้งงบประมาณและแจ้งเตือนค่าใช้จ่ายสำหรับ GPU, storage, egress และ log retention ก่อนเปิด worker ฝึกโมเดลต่อเนื่อง เพื่อควบคุมค่าใช้จ่ายที่เพิ่มตามปริมาณข้อมูล

5 3 3 แผนย้ายจากเครื่องพัฒนาไป Cloud

ระยะที่ 1 สร้างและเผยแพร่ container images เข้า private registry จัด configuration ต่อ environment ผ่าน secret manager และเพิ่ม infrastructure-as-code เพื่อสร้าง network, services, database, queue, buckets และ access policies ให้ทำซ้ำได้

ระยะที่ 2 ตั้ง staging environment แล้วย้าย schema และข้อมูลตัวอย่างที่ไม่มีข้อมูลระบุตัวบุคคล ทดสอบการสมัคร/เข้าสู่ระบบ บันทึกข้อมูล อัปโหลดภาพ ส่งคิว ประมวลผล อ่านผล หมดยา/ลบไฟล์ และกู้คืน backup ก่อนย้ายข้อมูลผู้ใช้งานจริง

ระยะที่ 3 ย้ายข้อมูลจริงด้วยแผน downtime หรือ controlled cutover ที่กำหนดไว้ล่วงหน้า ตรวจสอบจำนวน records, object keys, consent และความสัมพันธ์ระหว่างข้อมูลก่อนเปิด traffic ที่ละส่วน เก็บระบบเดิมไว้เป็นทางย้อนกลับจนกว่าจะยืนยันความครบถ้วนและความเสถียร

5 3 4 การตรวจรับและดูแลระบบบน Cloud

ทำ end-to-end tests ครอบคลุม frontend → API → PostgreSQL/Redis/ARQ → worker → object storage → การอ่านผล รวมทั้งกรณี consent ถูกถอน งานล้มเหลวและไฟล์หมดยา/ลบไฟล์ เพิ่ม monitoring สำหรับ latency, queue depth, worker failures, database/storage health, UV snapshot freshness และ model version พร้อม alert ที่ผู้ดูแลรับผิดชอบได้

ทดสอบการ restoreฐานข้อมูลและไฟล์จาก backup เป็นระยะ ตรวจสอบสิทธิ์เข้าถึง log และ artifacts และกำหนด retention/deletion ให้ครอบคลุมทั้ง Aphrodize, MLflow และ Label Studio

ก่อนรับข้อมูลผู้ใช้งานจริง การย้ายขึ้น Cloud ไม่ได้ทดแทนการทดสอบความแม่นยำ การตรวจรับโมเดล หรือการทบทวนความยินยอม

Acne signal และแบบบันทึกสียังคงอยู่นอกขอบเขต UI ปัจจุบัน

จึงไม่รวมเป็นงานเปิดใช้งานในการย้ายระบบครั้งนี้

เอกสารอ้างอิง

- [1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," arXiv:1505.04597, 2015.
[Online]. Available: <https://arxiv.org/abs/1505.04597>. [Accessed: Oct. 7, 2026].
- [2] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs," arXiv:1907.06724, 2019. [Online]. Available: <https://arxiv.org/abs/1907.06724>. [Accessed: Oct. 7, 2026].
- [3] NIST/SEMATECH, "Linear Least Squares Regression," e-Handbook of Statistical Methods, Sec. 4.1.4.1. [Online]. Available: <https://www.itl.nist.gov/div898/handbook/pmd/section1/pmd141.htm>. [Accessed: Oct. 7, 2026].
- [4] Redis Ltd., "Redis Data Types," Redis Documentation. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>. [Accessed: Oct. 7, 2026].
- [5] ARQ Project, "ARQ: Async Job Queue," arq v0.28.0 Documentation. [Online]. Available: <https://arq-docs.helpmanual.io/>. [Accessed: Oct. 7, 2026].
- [6] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Oct. 7, 2026].
- [7] SQLAlchemy Project, "SQLAlchemy 2.0 ORM Documentation." [Online]. Available: <https://docs.sqlalchemy.org/en/20/orm/index.html>. [Accessed: Oct. 7, 2026].
- [8] MinIO, "Install and Deploy MinIO: MinIO Object Storage for Linux," MinIO Documentation. [Online]. Available: <https://min.io/docs/minio/linux/operations/installation.html>. [Accessed: Oct. 7, 2026].
- [9] FastAPI Project, "Tutorial—User Guide," FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/tutorial/>. [Accessed: Oct. 7, 2026].
- [10] Vercel, "App Router," Next.js Documentation. [Online]. Available: <https://nextjs.org/docs/app>. [Accessed: Oct. 7, 2026].
- [11] MLflow Project, "MLflow Tracking," MLflow Documentation. [Online]. Available: <https://mlflow.org/docs/latest/tracking/>. [Accessed: Oct. 7, 2026].
- [12] HumanSignal, "Project Components," Label Studio Documentation. [Online]. Available: https://labelstud.io/guide/project_components.html. [Accessed: Oct. 7, 2026].
- [13] statsmodels Developers, "SARIMAX: Seasonal Autoregressive Integrated Moving Average with Exogenous Regressors," statsmodels Documentation. [Online]. Available: <https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html>. [Accessed: Oct. 7, 2026].
- [14] Docker Inc., "Docker Compose," Docker Documentation. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Oct. 7, 2026].
- [15] OpenTelemetry Authors, "Observability Primer," OpenTelemetry Documentation. [Online]. Available: <https://opentelemetry.io/docs/concepts/observability-primer/>. [Accessed: Oct. 7, 2026].
- [16] Prometheus Authors, "First Steps with Prometheus," Prometheus Documentation. [Online]. Available: https://prometheus.io/docs/introduction/first_steps/. [Accessed: Oct. 7, 2026].
- [17] MLflow Project, "Model Registry Workflows," MLflow Documentation. [Online]. Available: <https://mlflow.org/docs/latest/ml/model-registry/workflow/>. [Accessed: Oct. 7, 2026].
- [18] HumanSignal, "Export Annotations and Data," Label Studio Documentation. [Online]. Available: <https://labelstud.io/guide/export.html>. [Accessed: Oct. 7, 2026].

[19] TEMIS, "UV Index and UV Dose: Data Product Description," TEMIS/KNMI. [Online]. Available: <https://www.temis.nl/uvradiation/product/>.
[Accessed: Oct. 7, 2026].

ภาคผนวก หลักฐานและตำแหน่งโค้ด

หลักฐานการตรวจสอบที่รันซ้ำได้

คำสั่งและผลด้านล่างคงจากต้นฉบับ เป็นหลักฐานที่รายงานไว้
ไม่ใช่ผลการรันใหม่ในวันที่จัดรูปแบบเอกสาร

การตรวจสอบ	คำสั่งที่ใช้	ผล
Backend tests	<pre>python -m pytest tests/test_quality_gate.py tests/test_inference_worker.py tests/test_analysis_artifacts.py tests/test_landmark_rois.py tests/test_personal_outline.py tests/test_daily_health_personal_forecast.py tests/test_api_auth.py -q</pre>	32 passed, 1 warning, 0 failed; 15.70 วินาที
Frontend lint	<pre>npm run lint</pre>	ผ่าน ไม่มี lint error
Frontend type check	<pre>npx tsc --noEmit</pre>	ผ่าน ไม่มี TypeScript error

หัวข้อ	ไฟล์หรือองค์ประกอบที่อ้างถึง
บัญชีและโปรไฟล์	<pre>frontend/src/components/app-navigation.tsx frontend/src/app/api/auth/ frontend/src/app/signup/page.tsx frontend/src/app/profile/page.tsx</pre>
Daily health	<pre>frontend/src/app/clients/daily-health-history-panel.tsx backend/api/v1/routes/daily_health.py</pre>

หัวข้อ	ไฟล์หรือองค์ประกอบที่อ้างถึง
ROI / outline	tests/test_landmark_rois.py tests/test_personal_outline.py backend/workers/inference_worker.py
หน้า Home และ design	frontend/src/components/home-hero.tsx frontend/src/app/home.css frontend/src/app/design-system.css frontend/DESIGN.md
Progress 2 tests	tests/test_quality_gate.py tests/test_inference_worker.py tests/test_analysis_artifacts.py

เอกสารประกอบของโครงการ

Repository github.com/PacharaponK/Aphrodize

[docs/architecture/Component-Flows.md](#)

[docs/ai/Human-Review.md](#)

[docs/uv-model-workflow.md](#)

[frontend/DESIGN.md](#)